

Music Genre Classification using Deep Learning

COMP6252 Coursework 1 – University of Southampton

Akash

ECS User ID: 37603035

aalg25@soton.ac.uk

1 Introduction

This report presents the implementation and evaluation of six neural network classifiers for the GTZAN music genre dataset [1], covering ten genres: blues, classical, country, disco, hip-hop, jazz, metal, pop, reggae, and rock. Four image-based classifiers (Net1–Net4) operate on MEL spectrogram images, while two audio-based classifiers (Net5–Net6) operate on pre-extracted audio features. All models are implemented in PyTorch and trained under consistent experimental conditions to enable fair comparison.

2 Dataset and Preprocessing

The GTZAN dataset contains 1,000 audio clips (100 per genre), each 30 seconds long. Two data modalities are used: PNG MEL spectrogram images (`images_original/`) for Net1–Net4, and the pre-extracted feature CSV file (`features_3_sec.csv`) for Net5–Net6.

2.1 Image Pipeline (Net1–Net4)

Spectrogram images are loaded using `torchvision.datasets.ImageFolder` and pre-processed with the following transforms: `resize` to 180×180 pixels using `torchvision.transforms.Resize`, `conversion to tensor`, and `normalisation with mean and std of 0.5 across all three channels`. The dataset is randomly split using `torch.utils.data.random_split` with a fixed seed (42) into training (70%, 700 samples), validation (20%, 200 samples), and test (10%, 100 samples) sets. A batch size of 32 is used across all image models.

2.2 Audio Pipeline (Net5–Net6)

The pre-extracted `features_3_sec.csv` file is used, which contains 57 carefully engineered audio features (MFCCs, chroma, spectral centroid, zero-crossing rate, etc.) computed over 3-second chunks, yielding 10 chunks per 30-second track and 10,000 rows in total. Features are standardised using `StandardScaler` and grouped into sequences of

10 timesteps per song, producing 1,000 sequences of shape (10, 57). This design gives the LSTM a meaningful temporal structure to learn from. The same 70/20/10 split is applied with seed 42.

3 Neural Network Architectures

3.1 Net1 – Fully Connected Network

Net1 is a fully connected (FC) network with two hidden layers. `AdaptiveAvgPool2d(16, 16)` is applied before the FC layers, reducing the $180 \times 180 \times 3$ input from 97,200 to 768 features. Without this reduction, the weight matrix is intractably large and gradients vanish immediately. The architecture is: `AdaptiveAvgPool` $\rightarrow$ `Flatten` $\rightarrow$ `Linear(768, 512)` $\rightarrow$ `BatchNorm1d` $\rightarrow$ `ReLU` $\rightarrow$ `Dropout(0.4)` $\rightarrow$ `Linear(512, 256)` $\rightarrow$ `BatchNorm1d` $\rightarrow$ `ReLU` $\rightarrow$ `Dropout(0.4)` $\rightarrow$ `Linear(256, 10)`. Kaiming (He) initialisation is applied to all linear layers to improve gradient flow. Trained with Adam, $\text{lr}=10^{-4}$, weight decay= 10^{-4} .

3.2 Net2 – Convolutional Neural Network

Net2 follows the architecture in Figure 1 of the brief: two convolutional blocks each followed by `MaxPool`, then a fully connected head with 256 output features. `AdaptiveAvgPool2d(4, 4)` is applied before the classifier, reducing spatial dimensions from 45×45 to 4×4 and cutting FC parameters from 524,288 to 4,096, significantly speeding up training. Trained with Adam, $\text{lr}=10^{-4}$, weight decay= 10^{-4} .

Block 1: `Conv(3 $\rightarrow$ 32)` $\rightarrow$ `ReLU` $\rightarrow$ `Conv(32 $\rightarrow$ 64)` $\rightarrow$ `ReLU` $\rightarrow$ `MaxPool(2 $\times$ 2)`

Block 2: `Conv(64 $\rightarrow$ 128)` $\rightarrow$ `ReLU` $\rightarrow$ `Conv(128 $\rightarrow$ 256)` $\rightarrow$ `ReLU` $\rightarrow$ `MaxPool(2 $\times$ 2)`

Classifier: `AdaptiveAvgPool(4 $\times$ 4)` $\rightarrow$ `Flatten` $\rightarrow$ `Linear(4096, 256)` $\rightarrow$ `ReLU` $\rightarrow$ `Dropout(0.5)` $\rightarrow$ `Linear(256, 10)`

3.3 Net3 – CNN with Batch Normalisation

Net3 has the same architecture as Net2 but with `BatchNorm2d` inserted after every convolutional layer, before the activation. Batch normalisation normalises activations within each mini-batch, reducing internal covariate shift and stabilising training dynamics. All other hyperparameters remain identical to Net2.

3.4 Net4 – CNN with Batch Normalisation and RMSProp

Net4 is architecturally identical to Net3 but uses **RMSProp** instead of Adam. RMSProp adapts per-parameter learning rates by dividing by a running average of squared gradients, leading to different convergence behaviour on small datasets. Learning rate is 10^{-4} with weight decay 10^{-4} .

3.5 Net5 – Bidirectional LSTM

Net5 is a bidirectional LSTM classifier on sequences of pre-extracted audio features. Each song is represented as a sequence of 10 feature vectors of dimension 57. The architecture uses 2 LSTM layers with hidden size 128, bidirectional processing, and dropout of 0.3. Final hidden states from both directions are concatenated and passed to a classifier: $Linear(256, 128) \rightarrow ReLU \rightarrow Dropout(0.3) \rightarrow Linear(128, 10)$. Training uses Adam ($lr=10^{-3}$, weight decay= 10^{-4}), gradient clipping (max norm=1.0), a `ReduceLROnPlateau` scheduler, and early stopping with patience of 20 epochs.

3.6 Net6 – Bidirectional LSTM with GAN Augmentation

Net6 uses the same LSTM architecture as Net5 but is trained on a GAN-augmented dataset. A Conditional GAN (cGAN) is trained for 100 epochs to generate synthetic feature sequences conditioned on genre label. The Generator maps Gaussian noise and a label embedding to a (10, 57) feature sequence via three fully connected layers with batch normalisation and Tanh output. The Discriminator classifies sequences as real or fake using a three-layer network with LeakyReLU activations and Sigmoid output. After GAN training, synthetic samples equal in number to the real training set are generated and combined with real data, doubling the training set size. Both Generator and Discriminator are trained with Adam ($lr=2 \times 10^{-4}$, betas=(0.5, 0.999)).

4 Efficiency Improvements

Several optimisations were adopted to improve both training efficiency and model performance. `AdaptiveAvgPool2d`

was used in Net1–Net4 to reduce spatial feature map dimensions before fully connected layers, dramatically cutting parameter counts and training time — reducing FC inputs in Net2 from 524,288 to 4,096 (a $128 \times$ reduction). Kaiming (He) initialisation was applied in Net1 to ensure stable gradient magnitudes from the first epoch, avoiding the vanishing gradient problem present in the naive FC baseline. Gradient clipping (max norm = 1.0) was applied in Net5 and Net6 to prevent exploding gradients during LSTM backpropagation through time. A `ReduceLROnPlateau` scheduler was used in Net5 to adaptively reduce the learning rate when validation loss plateaued. Early stopping with patience was adopted for Net5 and Net6 to prevent overfitting and eliminate unnecessary computation. For the audio pipeline, pre-extracted CSV features were used instead of raw waveforms, eliminating computationally expensive feature extraction at runtime and ensuring high-quality, reproducible input representations.

5 Results

All models are evaluated on the held-out test set (100 samples). Table 1 summarises test accuracy across all six classifiers. Table 2 presents the overfitting analysis. Figure 1 shows the test accuracy comparison across all models. Figure 2 shows the train vs validation accuracy curves for each model, with the overfitting gap shaded in red. Figure 3 summarises the final overfitting gap per model.

Table 1: Test accuracy (%) for all classifiers. Net1–Net4 report 50/100 epoch results; Net5–Net6 use early stopping.

Model	Architecture	Optimiser	Test Acc (%)
Net1	FC	Adam	48.51 / 55.45
Net2	CNN	Adam	49.50 / 60.40
Net3	CNN+BN	Adam	70.30 / 70.30
Net4	CNN+BN	RMSProp	65.35 / 70.30
Net5	LSTM	Adam	78.22
Net6	LSTM+GAN	Adam	74.26

6 Discussion

6.1 FC vs CNN

As shown in Figure 1, Net5 achieves the highest test accuracy (78.2%), while image-based models cluster between 48–70%. Figure 2 reveals the train-validation dynamics for each model, and Figure 3 summarises the final overfitting gap.

Net1 achieves 55.5% at 100 epochs, which is above random chance (10%) but exhibits the most severe overfitting of all image-based models — a training accuracy of 80.8% against a validation accuracy of only 53.8%, giving a gap of 27.1%. This

Table 2: Overfitting analysis: train accuracy, validation accuracy, and gap at final epoch. Gap >20% flagged as severe overfitting.

Model	Test	Train	Val	Gap
Net1 (50ep)	48.51	62.52	51.76	10.8
Net1 (100ep)	55.45	80.83	53.77	27.1
Net2 (50ep)	49.50	48.21	36.18	12.0
Net2 (100ep)	60.40	58.66	50.25	8.4
Net3 (50ep)	70.30	71.96	63.82	8.1
Net3 (100ep)	70.30	84.69	71.36	13.3
Net4 (50ep)	65.35	70.82	61.81	9.0
Net4 (100ep)	70.30	82.12	68.84	13.3
Net5 (LSTM)	78.22	100.00	81.91	18.1
Net6 (LSTM+GAN)	74.26	98.70	71.36	27.3

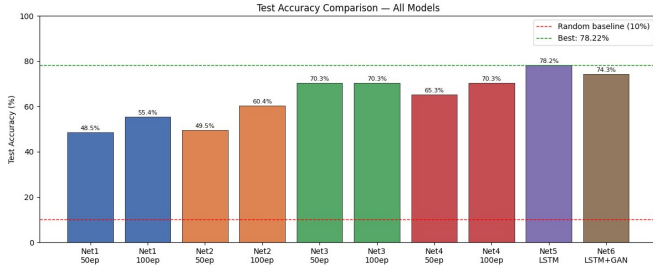


Figure 1: Test accuracy comparison across all models and epoch counts. The red dashed line marks the random baseline (10%) and the green dashed line marks the best result (Net5, 78.2%).

arises because FC networks treat every pixel independently and lack spatial inductive bias, forcing the model to memorise pixel-level co-occurrence patterns specific to the training set rather than learning generalisable spectral structure. The large number of parameters relative to the 700-sample training set exacerbates this tendency.

Net2 (plain CNN) achieves 60.4% at 100 epochs with a comparatively small gap of 8.4%, suggesting better generalisation than Net1. However, performance is limited by the absence of batch normalisation, which causes unstable gradient magnitudes across deeper layers, restricting how much the model can learn within 100 epochs.

6.2 Impact of Batch Normalisation

Batch normalisation in Net3 produces the largest performance jump of any single architectural change — 70.3% at both 50 and 100 epochs, compared to 49.5% and 60.4% for Net2. This demonstrates BN’s well-known benefits: normalised activations within each mini-batch reduce internal covariate shift and allow gradients to flow more consistently, enabling the model to converge to a better solution faster. Notably, Net3 achieves its

peak at 50 epochs with a modest gap of 8.1%, suggesting that BN acts as an effective implicit regulariser. At 100 epochs the gap widens to 13.3%, indicating the onset of overfitting once the model saturates the training distribution.

6.3 Adam vs RMSProp

Comparing Net3 (Adam) and Net4 (RMSProp) on identical architectures reveals that RMSProp converges more slowly — Net4 achieves only 65.3% at 50 epochs versus Net3’s 70.3%. However, both reach 70.3% at 100 epochs with similar overfitting gaps (13.3%). This suggests that on this dataset and scale, the choice of adaptive optimiser does not significantly affect the final generalisation capability, only the speed of convergence. Adam’s first-moment accumulation allows it to converge faster in early epochs, while RMSProp’s absence of momentum makes it more conservative.

6.4 Why Models Overfit

Overfitting is a recurring challenge across all six models and is attributable to several structural factors. The GTZAN dataset contains only 1,000 samples total, of which 700 are used for training — an extremely small dataset for deep learning. With this few examples, models with large parameter counts (Net1, Net5, Net6) can easily memorise the training distribution rather than learning generalisable features. Net1 at 100 epochs is the clearest example, with a 27.1% gap between training and validation accuracy.

For the LSTM models (Net5, Net6), the tendency to overfit is even more pronounced. Net5 reaches 100% training accuracy with an 18.1% gap, and Net6 reaches 98.7% with a severe 27.3% gap. In the case of Net6, the GAN-generated synthetic data, while doubling the training set size, does not introduce genuine diversity — the generator learns to produce samples from the same learned distribution, which can actually reinforce overfitting rather than mitigating it. This is evidenced by Net6 performing worse than Net5 on test accuracy (74.3% vs 78.2%) despite having more training data, suggesting the generated samples shifted the learned decision boundaries.

Regularisation techniques including Dropout (0.3–0.5), L2 weight decay, gradient clipping, batch normalisation, and early stopping were applied to limit overfitting. These were effective for Net2–Net4, which maintained gaps below 15%. For the LSTM models, the combination of high model capacity and a small dataset means some degree of overfitting is structurally unavoidable without additional real data or stronger regularisation such as variational dropout.

6.5 Audio Features vs MEL Spectrograms

Net5 (78.2%) and Net6 (74.3%) significantly outperform all image-based models. This difference is not purely architectural

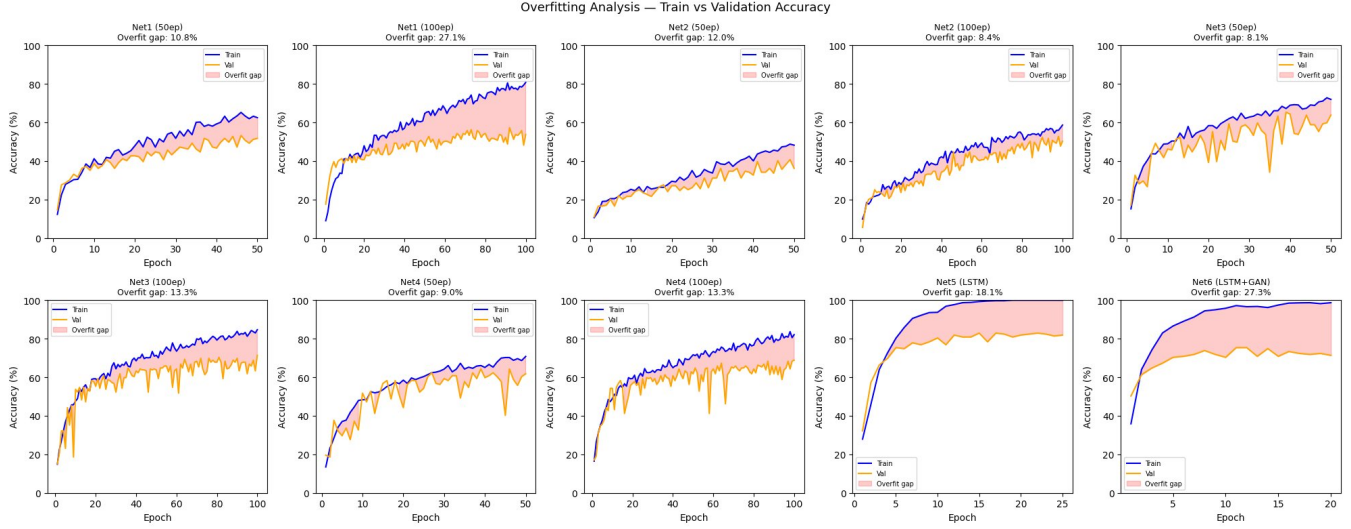


Figure 2: Train vs validation accuracy curves for all 10 model runs. The red shaded region represents the overfitting gap between training and validation accuracy. Larger shaded areas indicate more severe overfitting.

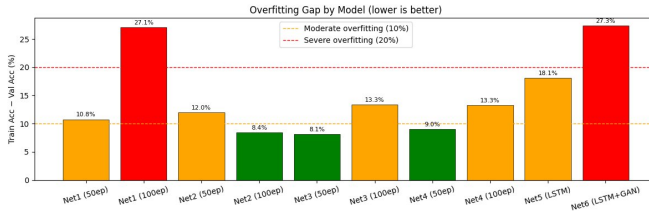


Figure 3: Final overfitting gap (Train Acc – Val Acc) per model. Green bars indicate acceptable generalisation (<10%), orange bars indicate moderate overfitting (10–20%), and red bars indicate severe overfitting (>20%).

— it is primarily a consequence of the *quality and compactness of the input representation*.

The MEL spectrogram images used by Net1–Net4 are high-dimensional 2D visual representations ($180 \times 180 \times 3 = 97,200$ raw values) that encode audio information implicitly through pixel intensities. While CNNs are well-suited to learning spatial patterns from images, music genre is determined by abstract properties such as rhythmic regularity, harmonic structure, timbral texture, and temporal dynamics — none of which are directly encoded as local spatial features in a spectrogram. The CNN must therefore learn to extract these higher-level audio concepts from raw pixel data, which is a much harder learning problem, particularly on a dataset of only 700 training images.

In contrast, the `features_3_sec.csv` pre-extracted features provide 57 carefully engineered audio descriptors per 3-second chunk — including MFCCs (which capture timbral texture), chroma features (which capture harmonic content), spectral centroid and bandwidth (which capture brightness and texture), and zero-crossing rate (which captures percussive-

ness). These features are the result of decades of audio signal processing research and directly encode the acoustic properties most discriminative for genre classification. The LSTM therefore receives inputs that are already semantically meaningful, requiring far less learning to separate genres effectively.

In other words, the image pipeline asks the CNN to rediscover what audio researchers have known for decades, while the CSV pipeline provides those findings directly. This gives the LSTM a substantial head start, explaining the ~ 18 percentage point accuracy advantage over the best CNN model.

6.6 Overall

Net5 (LSTM, 78.2%) achieves the best test accuracy overall. Among image-based models, Net3 and Net4 perform equally best at 70.3% (100 epochs). The results confirm that pre-extracted audio features are a more effective input modality than raw MEL spectrograms for this dataset and model scale, and that batch normalisation is the single most impactful architectural improvement for CNN-based classifiers.

7 Reflection: Batch Normalisation in Deep Learning

7.1 Why the Topic is Important

Batch normalisation (BN), introduced by Ioffe and Szegedy [2], is one of the most impactful advances in deep learning. Before BN, training deep networks was extremely sensitive to weight initialisation and learning rate choice. Small perturbations in early layers would cascade into large distribution

shifts in later layers — a phenomenon known as *internal covariate shift*. BN addresses this by normalising layer inputs within each mini-batch to have zero mean and unit variance, then applying learnable scale (γ) and shift (β) parameters. This allows networks to train with higher learning rates, reduces dependence on careful initialisation, and acts as a mild regulariser. The practical importance of BN is directly observable in this work: Net3 (with BN) outperforms Net2 (without BN) by approximately 21 percentage points at 100 epochs, despite being architecturally identical otherwise. Furthermore, Net3’s overfitting gap at 50 epochs is just 8.1% compared to 12.0% for Net2, confirming BN’s regularising effect.

7.2 Current Deep Learning Technologies

Several BN variants have been developed to address its limitations. **Batch Normalisation** [2] normalises across the batch dimension and works well for CNNs with large batch sizes. **Layer Normalisation** normalises across features and is preferred for RNNs and Transformers where batch size may be 1. **Group Normalisation** divides channels into groups and normalises within each, remaining robust to small batch sizes. **Instance Normalisation** normalises per sample per channel and is commonly used in style transfer. **Spectral Normalisation** controls the Lipschitz constant of weight matrices and is widely used in GANs to stabilise discriminator training.

7.3 Implementation Capability

Batch Normalisation (Net3, Net4) and **Dropout** as regularisation have been successfully implemented in this coursework. **Layer Normalisation** is understood conceptually and could be implemented using `nn.LayerNorm` for the LSTM models to potentially reduce the overfitting observed in Net5 and Net6. **Group Normalisation** and **Spectral Normalisation** present greater implementation challenges; the primary difficulty lies in understanding their interaction with gradient dynamics and tuning associated hyperparameters such as the number of groups or the Lipschitz bound.

7.4 Positive and Negative Impacts, and Future Vision

The positive impacts of normalisation technologies are substantial: they have enabled training of extremely deep networks (ResNet, VGG, BERT) that would otherwise be unstable, and have democratised deep learning by reducing reliance on expert hyperparameter tuning. However, BN introduces non-trivial complications — it creates a dependency between samples in a mini-batch, breaking the i.i.d. assumption and causing models to behave differently at train and test time. This can harm performance when batch sizes are very small or in online/streaming settings, as observed in experiments with small datasets like GTZAN. Looking forward, adaptive and learned normalisation

— where normalisation parameters are dynamically conditioned on input content rather than fixed per-layer — will become increasingly important in multimodal and generative models. Technologies such as Adaptive Instance Normalisation (AdaIN) already hint at this direction, and the integration of normalisation strategies with neural architecture search may eventually automate what is currently a manual design decision.

References

- [1] G. Tzanetakis and P. Cook, *Musical Genre Classification of Audio Signals*, IEEE Transactions on Speech and Audio Processing, 2002. 1
- [2] S. Ioffe and C. Szegedy, *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift*, ICML, 2015. 4, 5